

Introduction to Javascript

Prepared by

Md. Yeasin Ali [YAI]
Graduate Research Assistant,
Department of Computer Science and Engineering,
BRAC University

Contributors

Partha Bhoumik [PBK]
Lecturer,
Department of Computer Science and Engineering,
BRAC University

1.1 Javascript

➤ What is JavaScript ?

According to the official [documentation](#):

*“JavaScript (JS) is a lightweight, interpreted, or just-in-time compiled programming language with **first-class functions**. It is is a prototype-based, multi-paradigm, **single-threaded**, **dynamic** language, supporting object-oriented, imperative, and declarative (e.g. functional programming) styles”*

➤ Why is it important ?

1.2 Data types & Variables

JavaScript provides eight different data types which are:

1. string
2. boolean
3. number
4. object
5. undefined
6. null
7. [bigint](#)
8. [symbol](#)

JavaScript provides four different ways to declare a variable using:

1. var
2. let
3. const
4. Using nothing

1.3. Why var, let and const ?

keyword	const	let	var
global scope	No	No	Yes
functional scope	Yes	Yes	Yes
block scope	Yes	Yes	No
re-assign	No	Yes	Yes
re-declare	No	No	Yes

1.3.1 Global Scope

- Variables declared in the global scope are accessible from anywhere in the code, both inside and outside functions.

```
1  var globalVar = "CSE-446"; // This variable is in global scope
2
3  function globalScopeExample() {
4      // Accessing globalVar within the function
5      console.log(globalVar, "Accessing globalVar within the function");
6  }
7
8  globalScopeExample();
9  console.log(globalVar, "Accessing globalVar outside the function");
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\ASUS\Downloads\JS Workshop codes\Day-1> node "scopeExample.js"
CSE-446 Accessing globalVar within the function
CSE-446 Accessing globalVar outside the function
```

1.3.2 Functional Scope

- Variables declared within a function are only accessible within that function. They are local to the function and cannot be accessed from outside.

```
11 function functionalScopeExample() {
12     var localVar = "CSE-446"; // This variable is in functional scope
13     console.log(localVar, "Accessing localVar within the function");
14 }
15
16 functionalScopeExample();
17
18 // Trying to access localVar outside the function will result in an error
19 console.log(localVar);
20
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\ASUS\Downloads\JS Workshop codes\Day-1> node "scopeExample.js"
CSE-446 Accessing localVar within the function
C:\Users\ASUS\Downloads\JS Workshop codes\Day-1\scopeExample.js:19
console.log(localVar);
 ^
ReferenceError: localVar is not defined
 at Object.<anonymous> (C:\Users\ASUS\Downloads\JS Workshop codes\Day-1\scopeExample.js:19:13)
 at Module._compile (node:internal/modules/cjs/loader:1256:14)
 at Module._extensions..js (node:internal/modules/cjs/loader:1310:10)
 at Module.load (node:internal/modules/cjs/loader:1119:32)
 at Module._load (node:internal/modules/cjs/loader:960:12)
 at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:81:12)
 at node:internal/main/run_main_module:23:47

1.3.3 Block Scope

- Variables declared with **let** and **const** within blocks (typically enclosed by curly braces) have block scope. They are limited to that specific block and are not accessible outside of it.

```
1 // Example with let
2 if (true) {
3   let blockVar = "Blockchain in a Block"; // This variable is in block scope
4   console.log(blockVar);
5 }
6
7 // Trying to access blockVar outside the block will result in an error
8 console.log(blockVar);
9
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\ASUS\Downloads\JS Workshop codes\Day-1> node "scopeExample.js"
Blockchain in a Block
C:\Users\ASUS\Downloads\JS Workshop codes\Day-1\scopeExample.js:28
console.log(blockVar);
      ^
ReferenceError: blockVar is not defined
    at Object.<anonymous> (C:\Users\ASUS\Downloads\JS Workshop codes\Day-1\scopeExample.js:28:13)
    at Module._compile (node:internal/modules/cjs/loader:1256:14)
    at Module._extensions..js (node:internal/modules/cjs/loader:1310:10)
    at Module.load (node:internal/modules/cjs/loader:1119:32)
    at Module._load (node:internal/modules/cjs/loader:960:12)
    at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:81:12)
    at node:internal/main/run_main_module:23:47
```

```
30 // Example with const
31 function blockScopeExample() {
32   if (true) {
33     const blockConst = "Blockchain in a Block"; // This constant is also in block scope
34     console.log(blockConst);
35   }
36   // Trying to access blockConst outside the block will result in an error
37   console.log(blockConst);
38 }
39 blockScopeExample();
40
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\ASUS\Downloads\JS Workshop codes\Day-1> node "scopeExample.js"
Blockchain in a Block
C:\Users\ASUS\Downloads\JS Workshop codes\Day-1\scopeExample.js:37
  console.log(blockConst);
                ^
ReferenceError: blockConst is not defined
    at blockScopeExample (C:\Users\ASUS\Downloads\JS Workshop codes\Day-1\scopeExample.js:37:15)
    at Object.<anonymous> (C:\Users\ASUS\Downloads\JS Workshop codes\Day-1\scopeExample.js:39:1)
    at Module._compile (node:internal/modules/cjs/loader:1256:14)
    at Module._extensions..js (node:internal/modules/cjs/loader:1310:10)
    at Module.load (node:internal/modules/cjs/loader:1119:32)
    at Module._load (node:internal/modules/cjs/loader:960:12)
    at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:81:12)
    at node:internal/main/run_main_module:23:47
```

1.3.4 Examples of var, let and const ?

```
1  const constantStorage = "CSE446_Lab"
2  constantStorage = "Blockchain_lab"    // This is wrong you can not re-assign to a const type variable
3  console.log(constantStorage)
4
5  let letStorage = "CSE446_Lab"
6  letStorage = "Blockchain_lab"        // You can re-assign to a let type variable
7  console.log(letStorage)
8
9  let letStorage = "Js Workshop Day -1" // This is wrong you can not re-initialize a let type variable
10
11  var varStorage = "CSE446_Lab"
12  varStorage = "Blockchain_lab"        // You can re-assign to a var type variable
13  console.log(varStorage)
14
15  var varStorage = "Js Workshop Day -1" // You can re-initialize a var type variable
```

1.4 Operators

- ❖ There are different types of JavaScript operators.
 - Arithmetic Operators: `+`, `-`, `*`, `/`, `%`, `++`, `--` etc.
 - Assignment Operators: `=`, `+=`, `-=`, `*=`, `/=` etc.
 - Comparison Operators: `==`, `!=`, `===`, `!==`, `>=`, `>`, `<`, `<=` etc.
 - Logical Operators: `&&`, `||`, `!` etc.
 - Conditional(Ternary) Operator: **condition** `?` **exe_if_true** `:` **exe_if_false**

1.4.1 “==” Vs “===”

- ❖ “==” compares values for equality but performs **type coercion** if the values have different data types.

```
1 console.log(5 == "5"); // true (Type coercion: string "5" is converted to a number)
2 console.log(true == 1); // true (Type coercion: true is converted to 1)
3 console.log(null == undefined); // true (Both have similar values of "no value")
4
5 // Be careful with type coercion, it might lead to unexpected results
6 console.log(0 == false); // true (Type coercion: false is converted to 0)
```

- ❖ “===” It compares values for equality without performing type coercion. Both the value and the data type must match for the comparison to be true.

```
9 console.log(5 === "5"); // false (Data types are different: number vs. string)
10 console.log(true === 1); // false (Data types are different: boolean vs. number)
11 console.log(null === undefined); // false (Data types are different: null vs. undefined)
12
```

1.4.2 Ternary Operator Example-1

❖ Syntax: `condition ? exe_if_true : exe_if_false`

```
27 // Ternary Operator Example
28
29 let isSunny = true; // Initially set to true
30 let weatherMessage = isSunny ? "It's a sunny day!" : "It's not sunny today.";
31 console.log(weatherMessage);
32
33 // Now, change isSunny to false using a ternary operator
34 isSunny = isSunny ? false : true;
35
36 // Check the weatherMessage again
37 weatherMessage = isSunny ? "It's a sunny day!" : "It's not sunny today.";
38 console.log(weatherMessage);
```

PROBLEMS 2

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\ASUS\Downloads\JS Workshop codes\Day-1> node "operatorExample.js"
It's a sunny day!
It's not sunny today.
```

1.4.2 Ternary Operator Example-2

❖ Syntax: condition ? exe_if_true : exe_if_false

```
14 // Ternary Operator Example (manipulating a string)
15
16 let isLoggedIn = true; // Initially set to true
17 const greeting = `Hello, ${isLoggedIn ? "User" : "Guest"}`;
18 console.log(greeting); // Outputs: "Hello, User"
19
20 // Now, change isLoggedIn to false using another ternary operator
21 isLoggedIn = isLoggedIn ? false : true;
22
23 // Check the greeting again
24 const newGreeting = `Hello, ${isLoggedIn ? "User" : "Guest"}`;
25 console.log(newGreeting); // Outputs: "Hello, Guest"
```

PROBLEMS 2

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\ASUS\Downloads\JS Workshop codes\Day-1> node "operatorExample.js"
Hello, User
Hello, Guest
```

1.5 Conditional Statements

- Like any other languages, javascript uses conditional statements to perform different actions based on different conditions such as: **if, else, else if, switch**.

```
1  const price = 0
2
3  if( price === 0 ){
4      console.log('Price is 100 TK')
5  }
6  else if( price === 2 || price === 3 ){
7      console.log('Price is 200 TK')
8  }
9  else{
10     console.log('Sorry, we are out of price.')
11 }
```

Fig: if-else statement

```
1  const price = 0
2  switch (price) {
3      case 1:
4          console.log('price is 100 TK');
5          break;
6      case 2:
7      case 3:
8          console.log('price is 200 TK');
9          break;
10     default:
11         console.log('Sorry, we are out of price.~');
12 }
```

Fig: Switch statement

Problem 1:

Number Type Detector

Goal: To practice -

- variable declaration
- using operators
- taking actions using conditional statements

Scenario:

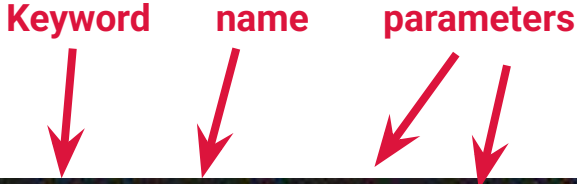
You are learning how javascript operators and if-else statements work. To understand it, you are creating a program which can detect **even**, **odd**, **fraction** and **zero** .

For easier understanding:

- If a number is evenly divisible by 2 with no remainder, then it is even. E.g: **num % 2 == 0**
- If a number is evenly divisible by 2 with remainder 1, then it is odd. E.g: **num % 2 == 1** or **negativeNum % 2 == -1**
- If neither even-odd nor zero, it is a fraction

1.6.1 Traditional Functions in js

- A JavaScript function is a block of code which performs particular task when invoked.
- In javascript, a function definition consists of:
 - A keyword **function**
 - Followed by the function name
 - Followed by a list of optional parameters enclosed by parentheses
 - The code that are executed by the function is wrapped by curly braces
 - A function can also return values
- Functions in javascript are **first-class**



```
function add(num1, num2){  
    const sum = num1 + num2  
    return sum  
}
```

1.6.2 Arrow Functions in js

- In addition to the traditional function, javascript(ES6) supports a new function expression called **Arrow Function**.

```
const add = (num1, num2) => {  
  const sum = num1 + num2  
  return sum  
}
```

Fig: Arrow Function

```
function add(num1, num2){  
  const sum = num1 + num2  
  return sum  
}
```

Fig: Traditional Function

For more information:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

Problem 2:

Number Type Detector Using Functional Approach

Goal:

- Understanding function declaration
- Invoke function

Scenario:

You are learning how javascript operators, if-else statements and **functions** work. To understand it, you are creating a program which can detect **even**, **odd**, **fraction** and **zero** .

For this, you have to **create a function** which will receive a number as parameter and the function will return the detected result.

For easier understanding, rules are given below:

- If the number is evenly divisible by 2 with no remainder, then the function will return "**even**".
E.g: **num % 2 == 0**
- If a number is evenly divisible by 2 with remainder 1, the function will return "**Odd**". E.g: **num % 2 == 1** or **negativeNum % 2 == -1**
- If neither even-odd nor zero, the function will return "**Fraction**"

1.7.1 Loops

- Like other programming languages, there are many different kinds of loops in javascript such as: **for**, **while**, **do.. While**, **for...in**, **for...of** etc. However, we will discuss the following loops:
 - **for**: loops through block of codes for a number of times defined by user.
 - **for...in**: loops through the properties/**key**/name of an object.
 - **for...of**: loops through the value of an iterable object.

1.7.2 Loops – Example

```
1  // for loop: loops through block of codes for a number of times defined by user.
2  for( let i = 0; i <= 5; ++i ){
3      console.log( i )
4  }
5
6  // for...in loop: loops through the properties/key of an object
7  const forInLoop = {
8      a: 5,
9      b: 10,
10     c: 15
11 }
12 for (const property in forInLoop) {
13     console.log( property );
14 }
15
16 // for...of loop: loops through the value of an iterable object.
17 const forOfLoop = [ 'CSE110', 'CSE111', 'CSE220' ]
18 for( const element of forOfLoop ){
19     console.log( element )
20 }
```

1.8.1 Array

- In javascript arrays are:
 - Zero indexed
 - Elements cannot be accessed using arbitrary string. Can only be accessed using nonnegative integers
 - Resizable
 - Can contain mix contents
 - Can be of shallow or deep

1.8.2 Array Methods

- There are many built-in array method in JS. Some of important array method that you should know are:
- **map:** iterate and return data
 - **forEach:** iterate data
 - **filter:** iterate and filter data
 - **slice:** create new shallow copy of array
 - **splice:** modify existing array by removing/adding new
 - **join:** create string by joining elements of array
 - **push:** add new element at the end of array
 - **pop:** remove last element of array
 - **shift:** remove first element of array
 - **unshift:** add new element at the beginning of array
 - **includes:** check if the array includes certain value and return true/false

1.9.1 Objects

- Objects are one of the js data types that contains data in a (key/name/property)->(value) pair.
- key/name/property - value pairs can contain any type of data.
- Multiple key/name/property - value pairs are separated using comma(,)
- We can access data using dot(.) or bracket([]) notation in js.
- Duplicate key/name/property will replace the older one

1.9.2 Objects

- An example of js object:

```
1  const myObj = {
2    propOne: 'I am number one',
3    propTwo: 19.5,
4    propThree: true,
5    4: 'My key/name/property is written using number',
6    propFive: [ 2, 4, 6 ],
7    propSix: {
8      childPropOne: 10,
9      childPropTwo: 'Child object Property'
10   }
11 }
12 // Access property value using dot notation
13 console.log( myObj.propOne ) // this will print 'I am number one'
14 console.log( myObj.propFive ) // this will print '[ 2, 4, 6 ]'
15 console.log( myObj.propSix.childPropTwo ) // this will print 'Child object property'
16 // console.log( myObj.4 ) // This will produce ERROR
17
18 //Access using bracket notation
19 console.log( myObj[ 4 ] ) // this will print 'My key/name/property is written using number'
20
21 const dynamicAccess = 'propOne'
22
23 console.log( myObj.propOne ) // will print 'I am number one'
24 console.log( myObj[ dynamicAccess ] ) // will print 'I am number one'
```

1.9.3 Methods of Objects

- `keys`: returns an array of keys of object
- `values`: returns the array of values of object
- `entries`: returns array of key, value pair arrays

Section 2:

String Manipulation

In this section, we will focus on string manipulation using most commonly used features and built-in methods.

For more details:

1. https://www.w3schools.com/jsref/jsref_obj_string.asp
2. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String

This section includes:

- Character access notation(bracket)
- length
- concat
- includes
- replace
- slice
- split

2.1 String Methods

➤ String methods Example:

```
1  let sampleStr = 'Hello Javascript'
2
3  // access character using bracket notation
4  sampleStr[ 0 ] // will return "H"
5  sampleStr[ 1 ] // will return "e"
6  sampleStr.length // will return 16
7
8  // concat: add string and return a new copy of string. Do not modify original string
9  // syntax: String.concat( trailingString1, trailingString2 ..... )
10 sampleStr.concat(' Lovers') // will return new string "Hello Javascript Lovers"
11
12 // includes: check the target string and return result in boolean
13 // syntax: includes( targetString )
14 sampleStr.includes('He') // will return true | This is case sensitive
15
16 // replace: will check the first argument and replace it with second one as new string object
17 // replace method don't modify the original string
18 // syntax: replace(targetString, stringToBeAdded)
19 sampleStr.replace('Javascript', 'World') // this will return a new string "Hello World"
20
21 // slice: extract part of the string from indexStart untill indexEnd and return a new one
22 // syntax: slice(indexStart, indexEnd)
23 sampleStr.slice( 1, 3 ) // will return "el"
24
25 // split: split the string matched with the passed argument and return a converted array from it
26 // syntax: split( seperatorString )
27 sampleStr.split(' ') // will return [ 'Hello', 'Javascript' ]
```

Problem: 2.1

Center character of a String

Goal:

- String manipulation
- Access character from string
- length

Problem

Find the character located at the middle of a string having a length of odd number.

Example:

sampleString = 'welcome to js'

Ans: e

Explanation:

Here “**sampleString**” holds a string of length 13, which is odd. Therefore, the ans is ‘e’, since it is located at the middle of the string.

Problem: 2.2

Concat ID

Goal:

- String manipulation
- `includes( targetString )`
- `replace( targetString, stringToBeAdded)`

Problem

You have a string: **"My student id is: Bolbona"**. Now, replace the word **"Bolbona"** with your **ID**. Next, check whether the student id contains **0** or not. If **zero** is found, print a message: **"ID contains zero"**. If not found print message: **"Not found"**. Use method: **"includes()"** to check zero. In addition, you are prohibited from using **var** and **let**

Example:

`sampleString = 'My Student id: Bolbone'`

After replacing, `sampleString` should be similar to:

"My Student id: 40404040"

Therefore, you have to print **"ID contains zero"**

Problem: 2.3

Create your character

Goal:

- String manipulation
- `slice(indexStart, indexEnd)`
- `split(separator, limit)`
- `concat( string1, string2, ..... )`

Problem

You have two anime superhero: **darkman** and **demon-slayer**. You have to create a new one which should be called **manslayer**.

- ★ You must solve this using all the methods provided in the Goal section of this problem.
- ★ You are prohibited from using “var” and “let”
- ★ You cannot create new variables directly using static value.

Section 3:

Array Manipulation

In this section, we will focus on array manipulation using most commonly used built-in methods and some notation.

For more details:

1. https://www.w3schools.com/JSREF/jsref_obj_array.asp
2. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

This section includes:

- **length**
- **concat**
- map
- forEach
- filter
- **slice**
- splice
- join
- push
- pop
- Shift
- unshift
- **includes**

3.1 Array Methods

- There are many built-in array method in JS. Some of important array method that you should know are:
- **map**: iterate and return data
 - **forEach**: iterate data
 - **filter**: iterate and filter data
 - **slice**: return a shallow copy of array
 - **splice**: modify **existing** array by adding/removing elements
 - **join**: create string by joining elements of array
 - **concat**: merge two or more arrays
 - **push**: add new element at the end of array
 - **pop**: remove last element of array
 - **shift**: remove first element of array
 - **unshift**: add new element at the beginning of array
 - **includes**: check if the array includes certain value and return true/false

3.2 Iterate array – map, forEach, filter

```
// forEach() → Iterates an array
const array = [1, 2, 3, 4]
array.forEach( (element, index) ⇒ console.log(`Element ${element}`))

// filter(): → Iterates an array and result is filtered array
const array = [1, 2, 3, 4]
const filter = array.filter(element ⇒ element%2)
// array ⇒ [1, 2, 3, 4]
// filter ⇒ [1, 3]

// map() → Iterates an array and result is new array
const array = [1, 2, 3, 4]
const mapArray = array.map( element ⇒ element * 2)
// mapArray ⇒ [2, 4, 6, 8]
```

3.3 Array – push, pop, shift, unshift

JS sample_3.3.js

```
1  const arr = [ 1, 2, 3 ]
2  console.log( "The original at the beginning : " + arr )
3
4  /*****
5   push( ) : This will modify the original array by inserting elements at the end of array
6   |         and then return the length of array
7   *****/
8  const afterPush = arr.push(4, 5)
9  console.log( "push(4, 5) returns: " + afterPush ) // This will print the length of array: 5
10 console.log( "The original is now : " + arr ) // This will print: [ 1, 2, 3, 4, 5 ]
11
12
13 /*****
14 unshift( ) : This will modify the original array by inserting elements at the beginning of array
15 |         and then return the length of array
16 *****/
17 const afterUnshift = arr.unshift( -1, 0 )
18 console.log( "unshift(-1, 0) returns: " + afterUnshift ) // This will print the length of array: 7
19 console.log( "The original is now : " + arr ) // This will print: [ -1, 0, 1, 2, 3, 4, 5 ]
20
21
22 /*****
23 pop( ) : This will modify the original array by removing last element of array
24 |         and then return that removed element
25 *****/
26 const afterPop = arr.pop()
27 console.log( "pop() returns: " + afterPop ) // This will print the last removed element of array: 5
28 console.log( "The original is now : " + arr ) // This will print: [ -1, 0, 1, 2, 3, 4 ]
29
30
31 /*****
32 shift( ) : This will modify the original array by removing the first element of the array
33 |         and then return that removed element
34 *****/
35 const afterShift = arr.shift()
36
37 console.log( "shift() returns: " + afterShift ) // This will print the first removed element of array: -1
38 console.log( "The original is now : " + arr ) // This will print: [ 0, 1, 2, 3, 4 ]
```

// push() → Insert an element at the end

```
const array = [1, 2, 3, 4];
array.push(12) // → [1, 2, 3, 4, 12]
```

// pop(): → Remove an element from the end

```
const array = [1, 2, 3, 4];
array.pop() // → result [1, 2, 3]
```

// unshift(): inserts an element in the beginning

```
const array = [1, 2, 3, 4];
array.unshift(7) // result [7, 1, 2, 3, 4]
```

// Shift(): → REMOVE FIRST ELEMENT

```
const array = [1, 2, 3, 4];
array.shift() // result [2, 3, 4]
```

3.4 Array – slice, splice

```
6  /*****
7  |   slice: return a new array(shallow copy) between a range without modifying the original array
8  |   syntax: slice( start, end )
9  |   *****/
10 var myArr = ["a", "b", "c", "d"]
11 const slicedData = myArr.slice(1, 3) // slicing data from index 1 untill index 3 and return a copy
12 console.log( slicedData ) // will print: [ 'b', 'c' ]
13
14 const sliceAll = myArr.slice() // will slice all data and ** RETURN A COPY **
15 console.log( sliceAll ) // will print: [ 'a', 'b', 'c', 'd' ]
16
17 |
18 /*****
19 |   splice: modify the original array by removing/adding existing array elements
20 |   syntax: splice(start, deleteCount, item1, item2, itemN)
21 |   *****/
22 var myArr = ["a", "b", "c", "d"]
23 const splicedData = myArr.splice(1, 2) // modify original array by removing 2 elements starting from index 1 and return the removed data.
24 console.log( splicedData ) // will print: [ 'b', 'c' ]
25 console.log( myArr ) // will print: [ 'a', 'd' ]
```

3.5 Array- length, concat, includes, join

```
1  /**
2  * length
3  * syntax: Array.length
4  */
5  var myArr = ["a", "b", "c", "d"]
6  console.log( myArr.length ) // will print: 4
7  myArr.length = 2 // original array gets modified
8  console.log( myArr ) // will print: [ 'a', 'b' ]
9
10 /**
11 * concat
12 * syntax: Array.concat( trailing Array )
13 */
14 const course = [ 'C', 'S', 'E' ]
15 const code = [ '4', '4', '6' ]
16 const final = course.concat( code ) // will be: [ 'C', 'S', 'E', '4', '4', '6' ]
17
18 /**
19 * join: bind the elements of array by adding the parameter value in between the elements and convert it into a string.
20 * syntax: Array.join( string )
21 */
22 const finalStr = final.join('') // will be CSE446
23
24 /**
25 * includes: iterate and check the target string exist or not. Return result as Boolean
26 * syntax: Array.includes( string )
27 */
28 const checkLowerCase = finalStr.includes( 'cse' ) // will return false
29 const checkUpperCase = finalStr.includes( 'CSE' ) // will return true
```

3.6 Question and Answer

- ★ You have an array ['CSE110', 'CSE111', 'CSE220']. Your task is to print the elements of the array. Which method should you use among **map**, **forEach** and **filter** ?
- ★ You have an array [1, 9, 7, 6, 10]. Your task is to **check and print elements** which are **greater than 5**. Which method should you use among **map**, **forEach** and **filter** ?
- ★ You have an array [1, 9, 7, 6, 10]. Your task is to **check and return all the elements** which are **greater than 5**. Which method should you use among **map**, **forEach** and **filter** ?

Problem – 3.1

largest number

Goal: To practice:

- `forEach/map/filter`
- Functions
- `condition`

Problem

You have an array of integers. Create a function that takes the array and find the largest element.

★ **Try to utilize topics mentioned in the goal section**

Problem – 3.2

Array modification

Goal: To practice:

- Array manipulation

Problem

You have an array

```
const sampleArr = [ 'Welcome' , 'to the' , ' JS' , 'workshop' ]
```

Now, convert the array into

```
const sampleArr = [ 'Welcome' , 'to the' , 'workshop' ]
```

★ **You cannot create any variable**

Problem – 3.3

Access Index

Goal: To practice:

- Use the best option from map/filter/forEach
- You can take help from other methods discussed earlier

Problem

Write a function that takes an array and return an array of the index of the value greater than 20

Example:

You have an array

```
const sampleArr = [ 1, 10, 7, 40, 50, 20 ]
```

Therefore, you have to return [3, 4].

★ **You cannot use “+” operator**